CO2 EXPERIMENT 12

12. Simulation and Performance Evaluation of Cross-Platform Edge AI Model Deployment using ONNX and PyTorch Mobile

1. AIM

To simulate and evaluate the performance of cross-platform Edge AI model deployment using ONNX Runtime and PyTorch Mobile by comparing inference latency, throughput, model size, memory usage, speedup and accuracy.

2. OBJECTIVES

1. To understand the concept of Edge AI and cross-platform model deployment.
2. To study the deployment of AI models using PyTorch and ONNX Runtime.
3. To simulate the performance of different AI deployment frameworks.
4. To calculate and compare inference latency and processing throughput.
5. To analyse model size and memory consumption.
6. To evaluate speedup and accuracy retention across different frameworks.
7. To generate graphs and identify an efficient framework for Edge AI applications.

3. PROCEDURE

1. Open Python IDLE or Google Colab and import the required libraries such as NumPy, Pandas and Matplotlib.
2. Define the simulation parameters, including the number of samples, baseline accuracy, model size and memory usage.
3. Define the deployment frameworks: PyTorch CPU, ONNX Runtime CPU and Mobile INT8.
4. Assign computational, storage, accuracy and memory factors for each framework.
5. Calculate inference latency using preprocessing, inference and postprocessing time.

\[T_{total}=T_{pre}+T_{infer}+T_{post} \]

6. Calculate throughput using:

\[Throughput=\frac{Number\ of\ Samples}{Total\ Time} \]

7. Calculate speedup by comparing the baseline PyTorch latency with other frameworks.
8. Calculate model storage reduction, memory reduction and accuracy retention.
9. Display all calculated performance values in a tabular format using Pandas.
10. Generate bar graphs to compare latency, throughput, speedup, model storage, memory usage and accuracy.
11. Analyse the results to identify the most efficient deployment framework.

program code

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


# 1. Simulation parameters

N = 10000

A_fp32 = 96.0

M_fp32 = 12.0

RAM_fp32 = 24.0


# Relative computational and storage factors


models = [&amp;quot;PyTorch CPU&amp;quot;, &amp;quot;ONNX Runtime CPU&amp;quot;, &amp;quot;Mobile INT8&amp;quot;]

compute_factor = np.array([1.0, 0.75, 0.55])

storage_factor = np.array([1.0, 0.90, 0.35])

accuracy_factor = np.array([1.0, 0.998, 0.990])
```

```
memory_factor = np.array([1.0, 0.85, 0.40])

# 2. Calculate performance using standard equations
T_pre = 0.3
T_post = 0.2
T_infer = 2.5 * compute_factor

latency = T_pre + T_infer + T_post
total_time = latency * N / 1000
throughput = N / total_time

speedup = latency[0] / latency
model_size = M_fp32 * storage_factor
memory = RAM_fp32 * memory_factor
accuracy = A_fp32 * accuracy_factor

storage_reduction = (M_fp32 - model_size) / M_fp32 * 100
memory_reduction = (RAM_fp32 - memory) / RAM_fp32 * 100
accuracy_retention = accuracy / A_fp32 * 100

# 3. Display results
results = pd.DataFrame({
&amp;quot;Framework&amp;quot;: models,
&amp;quot;Latency (ms)&amp;quot;: latency,
&amp;quot;Throughput (samples/sec)&amp;quot;: throughput,
&amp;quot;Speedup (x)&amp;quot;: speedup,
```

```
&quot;Model Size (MB)&quot;: model_size,

&quot;Storage Reduction (%)&quot;: storage_reduction,
&quot;Memory (MB)&quot;: memory,
&quot;Memory Reduction (%)&quot;: memory_reduction,
&quot;Accuracy (%)&quot;: accuracy,
&quot;Accuracy Retention (%)&quot;: accuracy_retention
})

print(&quot;SIMULATED PERFORMANCE EVALUATION&quot;)
print(results.round(2).to_string(index=False))

# 4. Plot comparisons
metrics = [
(&quot;Latency (ms)&quot;, &quot;Inference Latency&quot;),
(&quot;Throughput (samples/sec)&quot;, &quot;Throughput&quot;),
(&quot;Speedup (x)&quot;, &quot;Speedup&quot;),
(&quot;Model Size (MB)&quot;, &quot;Model Storage&quot;),
(&quot;Memory (MB)&quot;, &quot;Memory Usage&quot;),
(&quot;Accuracy (%)&quot;, &quot;Accuracy&quot;)
]

for col, title in metrics:
plt.figure(figsize=(7, 4))
plt.bar(models, results[col])
plt.title(title + &quot; Comparison&quot;)
```

```
plt.ylabel(col)
plt.xticks(rotation=10)
plt.grid(axis=&quot;y&quot;, alpha=0.3)
plt.tight_layout()
plt.show()

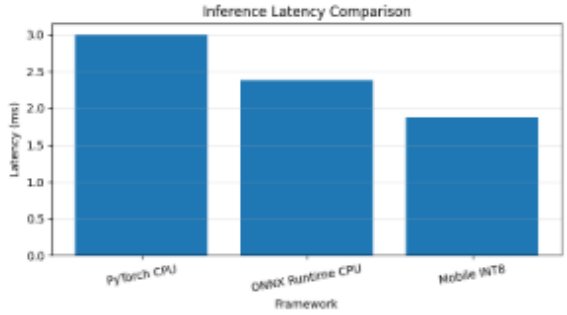
# 5. Display percentage improvements

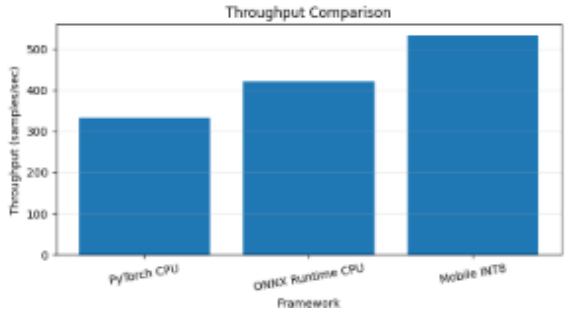
print(&quot;\nIMPROVEMENT OVER PYTORCH CPU&quot;)
for i in range(1, len(models)):
print(f&quot;\n{models[i]}&quot;)
print(f&quot;Latency improvement: {(latency[0]-latency[i])/latency[0]*100:.2f}%&quot;)
print(f&quot;Storage reduction: {storage_reduction[i]:.2f}%&quot;)
print(f&quot;Memory reduction: {memory_reduction[i]:.2f}%&quot;)
print(f&quot;Accuracy retention: {accuracy_retention[i]:.2f}%&quot;)
```

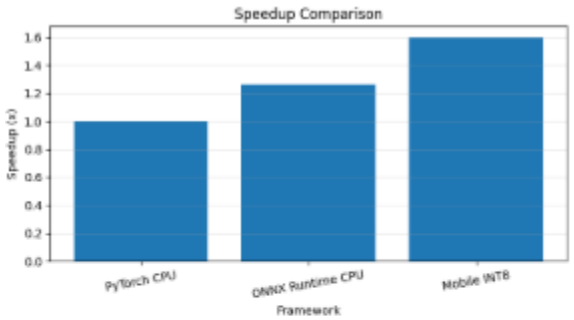
output

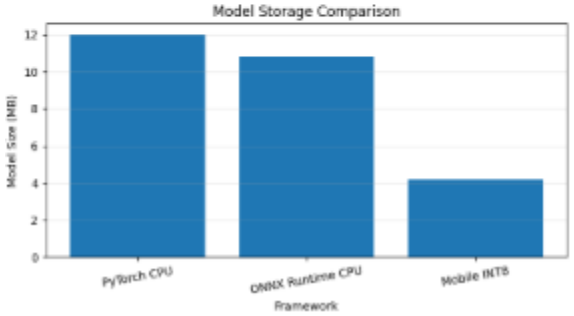
SIMULATED PERFORMANCE EVALUATION

Framework	Latency (ms)	Throughput (samples/sec)	Speedup (x)	Model Size (MB)	Storage Reduction (%)	Memory (MB)	Memory Reduction (%)	Accuracy (%)	Accuracy Retention (%)
PyTorch CPU	3.00	333.33	1.00	12.0	0.0	24.0	0.0	96.00	100.0
ONNX Runtime CPU	2.38	421.05	1.26	10.8	10.0	20.4	15.0	95.81	99.8
Mobile INT8	1.88	533.33	1.60	4.2	65.0	9.6	60.0	95.04	99.0









4. RESULT

The simulation and performance evaluation of cross-platform Edge AI model deployment using ONNX Runtime and PyTorch-based deployment approaches was successfully performed.

The results showed that the simulated Mobile INT8 approach achieved lower inference latency, reduced model storage and lower memory consumption compared to the baseline PyTorch CPU model. ONNX Runtime also demonstrated improved performance with reduced execution time.